ARAMO TECNOLOGIA

RV Pic Price

API DO SERVIDOR DA RV

Guia de integração

Os menores preços de venda de um produto perto de cada loja, pelo código de barras, para o backend do seu sistema.

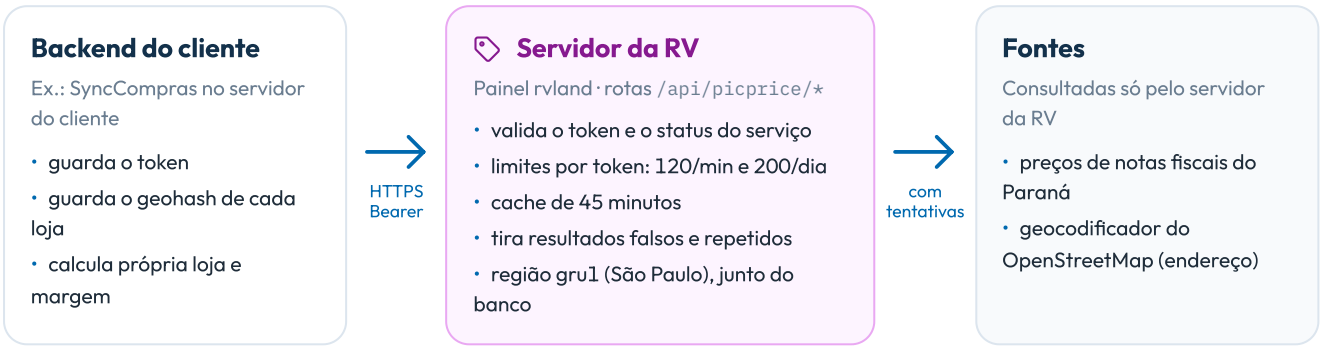
```
$ curl -H "Authorization: Bearer rvpp_…" \
  "…/api/picprice/ranking?gtin=7891010793463&lojas=001:6gmp0v90nd4t"
{ "estado": "OK", "lojas": [ { "loja": "001", "totalNaArea": 55,
  "precos": [ { "posicao": 1, "valor": 8.99, "distanciaKm": 1.5 }, … ] } ] }
```

Para desenvolvedores que consomem a API

Branch feat/rv-pic-price · 24/10/2026

O que a API faz

A API do **RV Pic Price** roda no servidor da RV e responde duas perguntas: **onde fica esta loja** (endereço para geohash) e **quais são os 10 menores preços de venda deste produto perto de cada loja** (pelo EAN). Os preços vêm de uma fonte pública de preços de notas fiscais do Paraná, consultada e filtrada pelo servidor da RV.



Nunca do navegador. A API é chamada pelo backend do cliente. O token não pode ir para o front, para o app nem para o HTML: quem tem o token consulta em nome do cliente.

O que a API entrega e o que fica com você

A API entrega

Geohash de um endereço do Paraná. Para cada loja: os 10 menores preços do EAN no raio, com valor, distância, há quanto tempo e a descrição da nota, e quantos preços existem na área. Sem nome nem endereço de concorrente.

Fica com o consumidor

Guardar o geohash das lojas. Saber o preço de venda e o custo do cliente. Reconhecer a própria loja na lista, posicionar o cliente e calcular margem e simulações. A página 9 mostra como o SyncCompras faz.

Como este guia está organizado

Página 3. Base URL e autenticação.

Página 4. `POST /api/picprice/geohash`

Página 5. `GET /api/picprice/ranking`

Página 6. Resposta do ranking, campo a campo.

Página 7. Estados, códigos HTTP e `GET /api/picprice/status`.

Página 8. Limites, cache e tempos.

Página 9. Regras da resposta e o que o consumidor calcula.

Página 10. Exemplo completo de consumo.

Base URL e autenticação

Todas as rotas ficam embaixo de `/api/picprice` no domínio do painel da RV. Toda requisição leva o token do cliente no header **Authorization**.

Ambiente	Base URL	Observação
Produção	`https://<domínio do painel da RV>`	Painel rvland na Vercel. Todas as funções rodam em gru1 (São Paulo), junto do banco (Supabase sa-east-1).
Teste local	`http://127.0.0.1:7790`	Servidor de teste do branch (`scripts/picprice-teste.ts`). Mesma lógica das rotas, com o token `rvpp_teste` e o status em memória.

```
# headers de toda requisição
Authorization: Bearer rvpp_Q2x5c0Fh…(43 caracteres base64url)
Content-Type: application/json        # só no POST
```

O token

- **Um por cliente**, vinculado ao **CNPJ completo da matriz** (14 dígitos) do cadastro do cliente no painel. Sem CNPJ completo, o painel não gera.
- **Formato:** `rvpp_` seguido de 32 bytes aleatórios em base64url.
- **Mostrado uma única vez**, na hora de gerar ou trocar. O painel guarda só o **SHA-256** do token e os 10 primeiros caracteres, para identificar na lista.
- **Último uso:** cada requisição aceita atualiza a data de último uso, visível no painel.
- **Quem gerencia:** a aba **Tokens RV Pic Price** do painel, com as permissões `picprice.ver` (ver tokens e status) e `picprice.gerenciar` (gerar, trocar, revogar, reativar e mudar o status do serviço). Cada ação entra na auditoria do painel.

Ciclo de vida

Ação no painel	Efeito na API
Gerar	Cria o token do cliente. Cole no backend do cliente; ele passa a responder `OK` .
Trocar	Gera um token novo para o mesmo cliente. O anterior deixa de funcionar na hora.
Revogar	Toda requisição com esse token recebe `401 TOKEN_INVALIDO` com `"motivo": "revogado"` e a mensagem **Serviço inválido - token revogado**. Afeta só aquele cliente.
Reativar	O mesmo token volta a funcionar, sem precisar colar de novo.

Ordem das verificações

Em ranking e geohash, antes de qualquer consulta: **1.** token (inexistente ou revogado: 401) → **2.** status do serviço (manutenção: 503) → **3.** limite por minuto (429 `LIMITE_EXCEDIDO` , não conta no dia) → **4.** conta a requisição no dia; se passou do limite, 429 `LIMITE_DIARIO` . Só então a rota valida os parâmetros e consulta as fontes, por isso uma requisição com parâmetro inválido também conta. O `GET /api/picprice/status` faz só os passos 1 e 2 e não conta (página 7).

Localizar uma loja

POST `/api/picprice/geohash`

Recebe o endereço de uma loja do Paraná e devolve o **geohash** de 12 caracteres usado no ranking. Chame **uma vez por loja** e guarde o resultado no servidor do cliente: o endereço da loja não muda a cada consulta.

Corpo

Campo	Tipo	Regra
`endereco`	texto	Obrigatório, ao menos 5 caracteres. Rua, número, bairro e cidade dão o melhor resultado.

Como o endereço é resolvido

O servidor da RV consulta o geocodificador do OpenStreetMap limitado ao retângulo do Paraná, pega o primeiro resultado com estado Paraná e converte a coordenada em geohash de 12 caracteres (célula de poucos centímetros; o raio em km é que define a área do ranking).

Requisição

```
curl -X POST "$BASE/api/picprice/geohash" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"endereco":"Rua XV de Novembro, 1000, Centro, Curitiba"}'
```

Resposta 200 (real, servidor de teste, 24/10/2026)

```
{
  "estado": "OK",
  "geohash": "6gkzqf298sdj",
  "lat": -25.4316937,
  "lon": -49.2729707,
  "endereco_encontrado": "Rua XV de Novembro - Centro - Curitiba - Paraná"
}
```

Respostas de erro desta rota

HTTP	estado	Quando
400	`REQUISICAO_INVALIDA`	Endereço vazio ou com menos de 5 caracteres. Mensagem: "Informe o endereço da loja."
404	`ENDERECO_NAO_ENCONTRADO`	Nenhum resultado no Paraná. Mensagem: "Endereço não encontrado no Paraná." Revise o endereço e tente de novo.
503	`FONTE_INDISPONIVEL`	O geocodificador falhou. Mensagem começa com "Geocodificador indisponível:".
401 · 503 · 429	token, manutenção, limite	As verificações comuns da página 3.

> **Confira o endereço encontrado**
>
> Mostre `endereco_encontrado` para quem configurou a loja. Se o geocodificador caiu na rua certa de outro bairro, o ranking vai olhar a região errada. Corrija o endereço enviado e localize de novo.

Ranking de preços por EAN

GET `/api/picprice/ranking`

Para cada loja enviada, os **10 menores preços de venda** do produto no raio em volta do geohash dela, do mais barato para o mais caro. Várias lojas vão numa só chamada; cada uma é consultada em paralelo.

Parâmetros da query

Nome	Padrão	Regra
`gtin`	obrigatório	EAN do produto. Só dígitos ficam; zeros à esquerda saem até restarem 8 dígitos. Precisa ter de 8 a 14 dígitos.
`lojas`	obrigatório	Lista separada por vírgula no formato `codigo:geohash`, ex.: `001:6gmp0v90nd4t,002:6gsbt0g0wusx`. Geohash de 5 a 12 caracteres do alfabeto base32 do geohash. Itens inválidos são ignorados; no máximo 30 lojas. O `codigo` é livre e volta igual na resposta.
`raio`	`3`	Raio em km, de 0,5 a 20. Fora disso, é ajustado para o limite.
`data`	`-1`	Janela da venda: `-1` últimos 2 meses · `0` último mês · `1` 15 dias · `2` semana · `3` 24 h · `4` 12 h · `5` 6 h · `6` última hora.

Requisição

```
curl -G "$BASE/api/picprice/ranking" \
  -H "Authorization: Bearer $TOKEN" \
  --data-urlencode "gtin=7891010793463" \
  --data-urlencode "lojas=001:6gmp0v90nd4t,004:6gmp05rwp4uf" \
  --data-urlencode "raio=3" --data-urlencode "data=-1"
```

Respostas de erro desta rota

HTTP	estado	Quando
400	`REQUISICAO_INVALIDA`	"EAN inválido." ou "Informe as lojas no formato codigo:geohash." (nenhuma loja válida).
503	`FONTE_INDISPONIVEL`	Nenhuma loja conseguiu resposta da fonte. Vem com `detalhe`, ex.: "a fonte devolveu só resultados falsos".
200	`OK` parcial	Algumas lojas responderam e outras não: as que falharam vêm em `lojasSemResposta`.

Resposta 503 real (servidor de teste, 24/10/2026)

```
{ "estado": "FONTE_INDISPONIVEL",
  "mensagem": "A fonte de preços não respondeu. Tente de novo em instantes.",
  "detalhe": "a fonte devolveu só resultados falsos" }
```

A resposta do ranking, campo a campo

Resposta 200 · formato exato do código. Preço, distância e "há" são de uma consulta real da loja 001 (EAN 7891010793463, 24/10/2026); datas e descrição são ilustrativas e a lista foi encurtada

```
{
  "estado": "OK",
  "gtin": "7891010793463",
  "raio": 3,
  "data": -1,
  "consultadoEm": "2026-10-24T15:38:12.481Z",
  "lojas": [
    {
      "loja": "001",
      "geohash": "6gmp0v90nd4t",
      "totalNaArea": 55,
      "precos": [
        { "posicao": 1, "valor": 8.99, "distanciaKm": 1.5, "ha": "há um dia",
          "dataHora": "2026-10-23T11:42:00.000Z", "descricao": "…descrição como veio na nota…" },
        { "posicao": 2, "valor": 9.99, "distanciaKm": 0.1, "ha": "há 12 minutos", … },
        { "posicao": 3, "valor": 9.99, "distanciaKm": 1.4, "ha": "há 17 horas", … },
        … até a posição 10
      ]
    }
  ],
  "lojasSemResposta": [],
  "uso": { "usado": 37, "limite": 200, "zeraEm": "2026-10-25T00:00:00-03:00" }
}
```

Campo	Tipo	Significado
`gtin` · `raio` · `data`	—	Os valores **realmente usados**, depois de normalizar a query. Mostre o raio desta resposta, não o que você pediu.
`consultadoEm`	ISO 8601	Hora da resposta do servidor da RV. Com cache, os preços podem ter sido buscados na fonte até 45 minutos antes.
`lojas[].loja` · `geohash`	texto	O código e o geohash que você mandou.
`lojas[].totalNaArea`	número	Quantos preços do produto a fonte tem na área e no período, **antes** de tirar repetidos e resultados falsos.
`precos[].posicao`	1 a 10	Posição contínua, do mais barato para o mais caro; empate de preço desempata pela menor distância.
`precos[].valor`	número	Preço de venda na nota, em reais.
`precos[].distanciaKm`	número	Distância do geohash da loja até o vendedor, arredondada em 2 casas.
`precos[].ha` · `dataHora`	texto	"há 3 horas" pronto para exibir, e a data e hora da venda (pode vir `null`).
`precos[].descricao`	texto	Descrição do produto como está na nota do vendedor. Útil para conferir embalagens diferentes com o mesmo EAN.
`lojasSemResposta`	lista	Códigos das lojas que não tiveram resposta da fonte nesta chamada.
`uso`	objeto	Consultas do token hoje: `usado` (já contando esta), `limite` diário e `zeraEm`, a próxima meia-noite de São Paulo (página 8). Vem em toda resposta OK de ranking e geohash. Use para avisar antes de estourar.

Nunca vem na resposta: nome, razão social, endereço ou código do estabelecimento concorrente. A API compara preço, não identifica quem vende.

Estados e o que fazer em cada um

Toda resposta é um objeto JSON com o campo **`estado`**. Quando não é `OK`, vem também **`mensagem`**, já em português e pronta para mostrar ao usuário. Decida pelo `estado`, não pelo texto.

estado	HTTP	Quando e campos extras	O que o consumidor deve fazer
`OK`	200	Consulta atendida. No ranking, pode ser parcial (`lojasSemResposta`).	Mostrar. Guardar em cache local por 20 a 30 minutos.
`TOKEN_INVALIDO`	401	`motivo` : `"revogado"` → "Serviço inválido - token revogado"; `"inexistente"` → "Serviço inválido - token inválido".	Mostrar a mensagem e **parar de consultar** até o suporte trocar ou reativar o token. Não repetir.
`MANUTENCAO`	503	Status do serviço em manutenção no painel. `mensagem` (padrão "Manutenção programada") e `previsao` (texto livre ou `null`).	Mostrar a mensagem e a previsão. Parar de consultar; tentar de novo só por ação do usuário ou depois de alguns minutos.
`LIMITE_EXCEDIDO`	429	Mais de 120 requisições no minuto para o mesmo token. "Muitas consultas em pouco tempo. Tente de novo em instantes." Não conta no limite do dia.	Esperar alguns segundos e repetir uma vez. Reduzir o paralelismo do pré-carregamento.
`LIMITE_DIARIO`	429	O token passou do limite do dia. "Limite diário de consultas atingido", com `limite` , `usado` e `zeraEm` no corpo e também em `uso` . No `/status` vem com HTTP 200.	Mostrar a mensagem e **não tentar de novo até a hora de** `zeraEm` . Parar o pré-carregamento.
`FONTE_INDISPONIVEL`	503	A fonte de preços (ou o geocodificador) não respondeu depois das tentativas internas. No ranking vem `detalhe` .	Mostrar "tente de novo". Repetir no máximo uma vez depois de alguns segundos; não insistir em laço nem em pré-carregamento.
`REQUISICAO_INVALIDA`	400	Parâmetro faltando ou fora do formato (EAN, lojas, endereço).	Erro do consumidor: corrigir a chamada. Não repetir igual. Item sem EAN nem deve ser consultado.
`ENDERECO_NAO_ENCONTRADO`	404	Só no geohash: nenhum resultado no Paraná.	Pedir um endereço mais completo.
`INALCANCAVEL`	—	**Não sai do servidor da RV.** É o estado que o consumidor cria quando não consegue falar com a API: sem conexão, tempo esgotado, HTTP sem JSON ou JSON sem `estado` .	Mostrar "Serviço inoperante/inalcançável". Tentar de novo por ação do usuário. O SyncCompras usa exatamente esse nome e esse texto.

Exemplos e a rota de status

429 · limite diário (ranking ou geohash)

```
{ "estado": "LIMITE_DIARIO",
  "mensagem": "Limite diário de consultas atingido",
  "usado": 200, "limite": 200,
  "zeraEm": "2026-10-25T00:00:00-03:00",
  "uso": { mesmos três campos } }
```

GET /api/picprice/status · 200

```
{ "estado": "OK",
  "uso": { "usado": 37, "limite": 200,
           "zeraEm": "2026-10-25T00:00:00-03:00" } }
```

`GET /api/picprice/status` , só com o header `Authorization` : confere o token e o status do serviço e devolve o uso do dia, **sem contar no limite**. Respostas: 200 com `OK` ou `LIMITE_DIARIO` ; 503 `MANUTENCAO` (com `mensagem` , `previsao` e `uso`); 401 `TOKEN_INVALIDO` . Use para testar a conexão, como o botão "Testar conexão" do SyncCompras faz.

Limites, cache e tempos

A fonte de preços é lenta e instável. O servidor da RV absorve boa parte disso com cache e tentativas, mas o consumidor precisa de tempos limite generosos e de uma regra clara de quando repetir.

Item	Valor	Detalhe
Limite por minuto	120 req/min	Por token, janela de 60 s contada na memória de cada instância da função. Acima disso: 429 LIMITE_EXCEDIDO, que não conta no limite do dia.
Limite por dia	200 req/dia (padrão)	Por token, configurável na aba de tokens do painel. Conta toda requisição aceita de ranking e de geohash, inclusive as atendidas pelo cache e as recusadas por parâmetro inválido. Não contam token inválido, manutenção, LIMITE_EXCEDIDO nem o `/status`. Zera à meia-noite de São Paulo. Acima disso: 429 LIMITE_DIARIO.
Cache do ranking	45 min	Por consulta à fonte (EAN, geohash, raio, janela e tamanho). Guardado no banco do painel (`picprice_cache`), vale para todos os clientes.
Resposta falsa da fonte	3 tentativas	A fonte às vezes devolve só itens inventados para atrapalhar robôs. O servidor detecta, espera 0,8 s, 1,6 s e 2,4 s e tenta de novo; isso nunca entra no cache. Se as três vierem falsas: 503.
Tentativas na fonte	até 4 rodadas	Cada requisição desiste em 15 s; sem resposta em 5 s sai uma cópia em paralelo (até 3 por rodada); entre rodadas, pausas de 1, 2,5 e 5 s (+ até 30%). Só repete erro de rede, tempo esgotado, 429 e 5xx.
Tamanho da busca	20, depois 50	Pede 20 resultados; se depois de limpar sobrarem menos de 10 e houver mais na área, pede 50.
Tempo máximo da rota	60 s	`maxDuration` das duas rotas na Vercel.
Região	gru1	São Paulo, para todas as funções do painel, ao lado do banco (Supabase sa-east-1). A fonte responde melhor, e às vezes só, de dentro do Brasil.

Tempo de resposta esperado

Situação	Tempo
Ranking já no cache do servidor da RV	0,1 a 0,3 s (medido no servidor de teste: 115 a 200 ms)
Ranking consultando a fonte	2 a 15 s por loja; as lojas rodam em paralelo
Fonte falhando até desistir	10 a 20 s, terminando em 503 FONTE_INDISPONIVEL
Geohash	cerca de 1 s

Recomendações para quem consome

- **Tempos limite:** conexão de 10 s; leitura de 40 s no geohash e 70 s no ranking. São os valores do SyncCompras (`PicPriceCliente`), acima dos 60 s da rota.
- **Repetir só o que é passageiro:** 429 e 503 FONTE_INDISPONIVEL, no máximo uma vez, com espera. Nunca repetir 400, 401 ou 404, nem MANUTENCAO em laço.
- **Cache local:** guarde respostas OK por 20 a 30 minutos por item e cotação. O SyncCompras/Hermes usa 25 minutos e só guarda respostas estáveis.
- **Pré-carregamento com moderação:** cada item pré-carregado gasta uma consulta do limite diário, mesmo vindo do cache. Carregue poucos itens à frente, no máximo 2 ou 3 de cada vez, e pare ao receber TOKEN_INVALIDO, MANUTENCAO, LIMITE_DIARIO, FONTE_INDISPONIVEL ou INALCANCAVEL.
- **Limite diário estourado:** trate o 429 LIMITE_DIARIO sem novas tentativas até a hora de `zeraEm`. Acompanhe `uso` nas respostas OK.
- **Uma chamada para várias lojas:** mande todas as lojas da cotação no mesmo `lojas=`; o servidor consulta em paralelo.

Regras da resposta e o que o consumidor calcula

O que o servidor da RV faz com os preços

1. **Tira os resultados falsos.** Item com id só de letras minúsculas, números e ponto **e** código de estabelecimento diferente de 43 caracteres é descartado. As duas marcas juntas evitam jogar fora item real.
2. **Junta repetidos.** Mesmo estabelecimento, mesma descrição e mesmo preço viram uma linha só.
3. **Ordena.** O mais barato primeiro; empate pela menor distância.
4. **Corta em 10** e numera de 1 a 10, sem repetir posição.
5. **Esconde o vendedor.** Sai só preço, distância, quando e descrição. O código do estabelecimento é usado internamente para deduplicar e nunca vai na resposta.

`totalNaArea` é o total informado pela fonte para a área e o período, contado antes da limpeza. Serve para dar a dimensão ("55 na área"), não para somar com a lista.

O que fica com o consumidor

A API não conhece o cliente além do token: não sabe o preço de venda, o custo nem qual vendedor da lista é a própria loja. O SyncCompras faz assim, e serve de **implementação de referência**:

1. Reconhecer a própria loja

As vendas da loja do cliente também entram na fonte. O SyncCompras (`PicPriceResource`) trata como própria loja o preço que está a **até 0,2 km** do geohash da loja e é **igual ao preço de venda atual** dela (diferença menor que meio centavo). Esse preço sai da lista de concorrentes e é descontado do total na área; as posições dos concorrentes são renumeradas.

2. Posicionar o cliente

Posição = 1 + quantidade de concorrentes com preço **menor** que o preço de venda do cliente. Em empate, o cliente fica na frente.

3. Margem

Mesma conta da consulta de preço do aramoMobile (SyncProdutos, `PostgresEjb.java` , linhas 963 a 978), também usada pelo HiperSync como lucro ÷ venda:

```
custo  = cade_ctnota + cade_debvenda + cade_ctvenda
margem = (preço − custo) ÷ preço × 100        # truncada em 2 casas
```

Nas simulações de preço, o débito de venda acompanha o preço no mesmo percentual de hoje e o custo de venda fica fixo em reais. O custo para manter a margem *m* num preço novo *P* é `P × (1 − m) − %débito × P − ctvenda` . O preço de venda usado é `cade_prvenda` , que é o promocional quando o item está em oferta.

Exemplo completo de consumo

O caminho que o SyncCompras segue, do token guardado até o ranking na tela. Use como roteiro para qualquer outro backend.

1. **Guarde a configuração no servidor do cliente.** URL base, token e raio. No SyncCompras: tabela `sync.sync_picprice` (`pcpr_ativo`, `pcpr_url`, `pcpr_token`, `pcpr_raio`). O token nunca é devolvido ao front; o front só vê os 4 últimos caracteres.
2. **Localize cada loja uma vez.** Chame o geohash com o endereço da loja e guarde o resultado. No SyncCompras: `sync.sync_empresas` (`semp_codempresa`, `semp_geohash`, `semp_lat`, `semp_lon`, `semp_endereco`, `semp_enderecoencontrado`).
3. **Consulte o ranking por demanda.** Só com EAN válido e só das lojas já localizadas, todas na mesma chamada.
4. **Calcule do seu lado** a própria loja, a posição do cliente e a margem (página 9), e trate o `estado` (página 7).

Passo 2 · localizar

```
curl -X POST "$BASE/api/picprice/geohash" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"endereco":"Rua Exemplo, 100, Centro, Pinhais"}'
# guardar: 001 → 6gmp0v90nd4t
```

Passo 3 · ranking das lojas da cotação

```
curl -G "$BASE/api/picprice/ranking" -H "Authorization: Bearer $TOKEN" \
  --data-urlencode "gtin=7891010793463" \
  --data-urlencode "lojas=001:6gmp0v90nd4t,002:6gsbt0g0wusx,003:6gkyzvz1y8dr,004:6gmp05rwp4uf" \
  --data-urlencode "raio=3" --data-urlencode "data=-1"
```

Passo 4 · tratamento no backend (pseudocódigo)

```
resp = GET ranking (conexão 10 s, leitura 70 s)
se falhou a conexão ou não veio JSON com "estado" → estado = INALCANCAVEL
conforme resp.estado:
  OK                → para cada loja: tirar própria loja (≤ 0,2 km e preço = venda),
                      renumerar, posição = 1 + concorrentes mais baratos, margens
  TOKEN_INVALIDO    → mostrar mensagem; parar consultas
  MANUTENCAO        → mostrar mensagem e previsão; parar consultas
  LIMITE_EXCEDIDO   → esperar e repetir 1 vez
  LIMITE_DIARIO     → mostrar mensagem; nada até zeraEm
  (testar conexão: GET /status, não conta no limite)
  FONTE_INDISPONIVEL → "tente de novo"; repetir no máximo 1 vez
  REQUISICAO_INVALIDA→ corrigir a chamada (não repetir)
```

Implementação de referência

Arquivo (SyncCompras 2.4.5.8)	Papel
`picprice/PicPriceCliente.java`	Chamadas HTTP à API (o "Testar conexão" usa o `/status`), tempos limite e o estado INALCANCAVEL.
`rest/resource/PicPriceResource.java`	Proxy para o front, própria loja, posição, margens e simulações.
`picprice/PicPriceCalculo.java`	Fórmulas de margem e de custo para manter a margem.
`repository/PicPriceRepository.java`	Configuração, lojas, EAN, preço de venda e custos.

Versão descrita: branch **feat/rv-pic-price** do painel rvland (027f64c), `lib/picprice/*` e `app/api/picprice/*`; tabelas `picprice_tokens` (com o limite e o contador do dia, migration 0019) e `picprice_cache`.